

Untitled diff

⊖ 141 removals

155 lines

```

1 | class TaskQueue:
2 |     # Create a bounded first-in, first-out
   |     (FIFO) queue for task management.
3 |     #
4 |     # Tasks are stored in the order they
   |     are received and processed in the
5 |     # same order. The queue enforces a
   |     maximum capacity to prevent unbounded
6 |     # growth and provides simple operations
   |     for adding, removing, and
7 |     # inspecting queued work.
8 |     #
9 |     # Args:
10 |     #     max_size (int): Maximum number of
   |     tasks that may be stored in the
11 |     #     queue at one time. Additional
   |     enqueue attempts are rejected
12 |     #     once this limit is reached.
13 |     def __init__(self, max_size=100):
14 |         self.queue = []           # Ordered
   |         collection of pending tasks.
15 |         self.max = max_size       # Upper
   |         bound on the number of queued tasks.
16 |
17 |     # Add a task to the back of the queue.
18 |     #
19 |     # The task is appended only if
   |     sufficient capacity remains.
20 |
21 |     # Args:
22 |     #     task: Object representing the
   |     work item to be queued.
23 |     #
24 |     # Returns:
25 |     #     bool:
26 |     #         True if the task was
   |     successfully queued.

```

⊕ 134 additions

155 lines

```

1 | class TaskQueue:
2 |     # Manage a bounded first-in, first-out
   |     (FIFO) collection of tasks.
3 |     #
4 |     # Tasks are processed in the same order
   |     they are added. The queue
5 |     # enforces a configurable capacity
   |     limit to prevent uncontrolled growth
6 |     # and provides basic operations for
   |     task insertion, retrieval, and
7 |     # inspection.
8 |     #
9 |     # Args:
10 |     #     max_size (int): Maximum number of
   |     tasks that may be stored at any
11 |     #     given time. Attempts to add
   |     tasks beyond this limit will fail.
12 |     def __init__(self, max_size=100):
13 |         self.queue = []           # Internal
   |         FIFO storage for pending tasks.
14 |         self.max = max_size       # Maximum
   |         number of tasks allowed in memory.
15 |
16 |     # Insert a task at the end of the
   |     queue.
17 |     #
18 |     # The task becomes the most recently
   |     queued item and will not be
19 |     # processed until all previously queued
   |     tasks have been removed.
20 |
21 |     # Args:
22 |     #     task: The task object to enqueue.
23 |     #
24 |     # Returns:
25 |     #     bool:
26 |     #         True if the task was
   |     successfully added.

```

```

27         # False if the queue is already
    full.

28     def enqueue(self, task):
29         if len(self.queue) >= self.max:

30             return False # Capacity
    reached; reject the new task.

31
32         self.queue.append(task)
33         return True
34
35     # Remove and return the next task to be
    processed.

36     #

37     # Because the queue follows FIFO
    semantics, the oldest queued task is
38     # always returned first.

39     #
40     # Returns:

41     # The task at the front of the
    queue, or None if no tasks are
42     # currently waiting.

43     def dequeue(self):
44         if not self.queue:

45             return None # No
    pending tasks are available.

46
47         return self.queue.pop(0)
48
49     # Check whether the queue currently
    contains any tasks.

50     #
51     # Returns:

52     # bool: True if the queue is empty,
    otherwise False.

53     def is_empty(self):
54         return len(self.queue) == 0
55
56     # Get the current number of queued
    tasks.

57     #
58     # Returns:

59     # int: Total number of tasks
    waiting in the queue.

60     def size(self):
61         return len(self.queue)
62

```

```

27         # False if the queue has
    already reached its capacity limit.

28     def enqueue(self, task):
29         if len(self.queue) >= self.max:

30             return False # Reject
    the task when no capacity remains.

31
32         self.queue.append(task)
33         return True
34
35     # Remove and return the task at the
    front of the queue.

36     #

37     # This operation preserves FIFO
    behavior by always returning the oldest
38     # pending task.

39     #
40     # Returns:

41     # The next task scheduled for
    processing, or None if the queue
42     # contains no tasks.

43     def dequeue(self):
44         if not self.queue:

45             return None # No queued
    work is available.

46
47         return self.queue.pop(0)
48
49     # Determine whether the queue currently
    contains any pending tasks.

50     #
51     # Returns:

52     # bool: True when the queue is
    empty; otherwise False.

53     def is_empty(self):
54         return len(self.queue) == 0
55
56     # Return the number of tasks currently
    waiting in the queue.

57     #
58     # Returns:

59     # int: Current queue length.

60     def size(self):
61         return len(self.queue)
62

```

```

63     # Inspect the next task scheduled for
    processing without removing it.

64     #

65     # This operation allows callers to view
    the upcoming task while leaving
66     # the queue unchanged.

67     #
68     # Returns:
69     #     The task at the front of the
    queue, or None if the queue is empty.
70     def peek(self):
71         if self.queue:
72             return self.queue[0]
73
74         return None
75
76
77 class RateLimiter:

```

```

78     # Implement a fixed-capacity sliding-
    window rate limiter.

79     #

```

```

80     # The limiter tracks timestamps of
    accepted requests and determines
81     # whether a new request can be allowed
    based on how many requests have
82     # occurred within the configured time
    window.

83     #
84     # Args:

```

```

85     #     limit (int): Maximum number of
    requests permitted during a single
86     #     rolling time window.
87     #     window (int | float): Duration of
    the rolling window in seconds.

88     def __init__(self, limit, window):

```

```

89         self.limit = limit        # Maximum
    requests allowed within the window.
90         self.window = window      # Rolling
    time window length in seconds.
91         self.calls = []           #
    Timestamps for previously accepted
    requests.

92

```

```

93     # Evaluate whether a request should be
    allowed at the specified time.

94     #

```

```

95     # Requests that fall outside the active
    window are discarded before the

```

```

63     # Retrieve the next task to be
    processed without removing it.

64     #

```

```

65     # Unlike dequeue(), this method leaves
    the queue unchanged and is useful
66     # when callers need to inspect the
    upcoming task before processing.

67     #
68     # Returns:
69     #     The task at the front of the
    queue, or None if the queue is empty.
70     def peek(self):
71         if self.queue:
72             return self.queue[0]
73
74         return None
75
76
77 class RateLimiter:

```

```

78     # Enforce a request limit using a
    sliding time-window algorithm.

79     #

```

```

80     # The limiter records timestamps for
    accepted requests and evaluates each
81     # new request against the number of
    requests that occurred within the
82     # configured rolling window.

83     #
84     # Args:

```

```

85     #     limit (int): Maximum number of
    requests permitted within a single
86     #     active window.
87     #     window (int | float): Window
    duration in seconds.

88     def __init__(self, limit, window):

```

```

89         self.limit = limit        # Maximum
    requests allowed per window.
90         self.window = window      # Length of
    the rolling window in seconds.
91         self.calls = []           # Accepted
    request timestamps.

92

```

```

93     # Determine whether a request should be
    accepted.

94     #

```

```

95     # Any timestamps that fall outside the
    current rolling window are removed

```

```

96     # current request is evaluated. If the
    number of remaining requests is
97     # below the configured limit, the
    request is accepted and its timestamp
98     # is recorded.

99     #
100    # Args:

101    #     timestamp (int | float): Time
    associated with the incoming request.

102    #
103    # Returns:
104    #     bool:

105    #         True if the request is
    permitted.
106    #         False if the rate limit has
    been reached.

107    def allow(self, timestamp):

108        # Remove request records that are
    no longer relevant to the current
109        # rolling window.

110        self.calls = [
111            t for t in self.calls
112            if timestamp - t < self.window
113        ]

114
115        if len(self.calls) < self.limit:
116            self.calls.append(timestamp)
117            return True
118
119        return False # Request
    exceeds the allowed request rate.
120
121
122    def retry(func, attempts=3, exceptions=
    (Exception,)):

123        # Execute a callable with automatic
    retry behavior.

124        #

125        # The target function is invoked
    repeatedly until it succeeds or the
126        # maximum number of attempts has been
    exhausted. Only exceptions whose
127        # types are included in the exceptions
    parameter trigger a retry.

128        #

```

```

96     # before evaluating the request. If the
    number of remaining requests is
97     # below the configured threshold, the
    request is accepted and recorded.

98     #
99    # Args:

100    #     timestamp (int | float):
    Timestamp associated with the request.

101    #
102    # Returns:
103    #     bool:

104    #         True if the request is
    allowed.
105    #         False if accepting the
    request would exceed the configured
106    #     request limit.

107    def allow(self, timestamp):

108        # Discard request records that are
    older than the active window and
109        # therefore no longer contribute to
    rate-limit calculations.

110        self.calls = [
111            t for t in self.calls
112            if timestamp - t < self.window
113        ]

114
115        if len(self.calls) < self.limit:
116            self.calls.append(timestamp)
117            return True
118
119        return False # Deny the
    request because the limit is met.
120
121
122    def retry(func, attempts=3, exceptions=
    (Exception,)):

123        # Execute a callable and automatically
    retry on recoverable failures.

124        #

125        # The function is invoked repeatedly
    until it succeeds or the maximum
126        # number of attempts has been reached.
    Only exceptions matching the
127        # supplied exception types trigger a
    retry; all other exceptions are
128        # propagated immediately.

129        #

```

129 | # Args:

130 | # func (callable): Function to execute.

131 | # attempts (int): Total number of execution attempts, including the

132 | # initial call.

133 | # exceptions (tuple[type]): Exception types that should be caught and

134 | # retried.

135 | #

136 | # Returns:

137 | # The value returned by func() if an attempt succeeds.

138 | #

139 | # Raises:

140 | # Any exception not listed in exceptions.

141 | # The final caught exception if all retry attempts fail.

142 | def retry(func, attempts=3, exceptions=(Exception,)):

143 | for i in range(attempts):

144 | try:

145 | return func() # Execute the target operation.

146 |

147 | except exceptions:

148 | # Re-raise the exception once the final retry has failed.

149 | if i == attempts - 1:

150 | raise

151 |

152 | # Defensive fallback. Under normal execution this statement is never

130 | # Args:

131 | # func (callable): Operation to execute.

132 | # attempts (int): Total number of execution attempts, including the

133 | # initial invocation.

134 | # exceptions (tuple[type, ...]): Exception types that should be

135 | # intercepted and retried.

136 | #

137 | # Returns:

138 | # Any: The value returned by the first successful invocation of func.

139 | #

140 | # Raises:

141 | # Any exception not listed in exceptions.

142 | # The final matching exception if all retry attempts fail.

143 | for i in range(attempts):

144 | try:

145 | return func() # Attempt to execute the target operation.

146 |

147 | except exceptions:

148 | # Once all configured attempts have been used, re-raise the most

149 | # recent exception so the caller can handle the failure.

150 | if i == attempts - 1:

151 | raise

152 |

153 | # Defensive fallback. Execution should never reach this point because